ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA	INFORME DE PRÁCTICA DE LABORATORIO

DEPARTAMENTO:	CIENCIAS DE LA COMPUTACION	CARRERA:	TECNOLOGIAS DE LA INFORMACION Y COMUNICACIÓN		
ASIGNATURA:	METODOLOGIA DE DESARROLLO DE SOFWARE	NIVEL:	3	FECHA:	10/11/2025
DOCENTE:	Ing. John Cruz	PRÁCTICA N°:		CALIFICACIÓN:	

Proyecto Grupal: Inventario de Zapatería

Corina Acosta y Erick Maigua

RESUMEN

Este informe documenta el análisis de requerimientos para el desarrollo de un sistema de gestión de E-commerce destinado a la Zapatería "Premium" de Santo Domingo. El propósito del proyecto es migrar las operaciones de venta e inventario, actualmente manuales, a una plataforma web robusta. El sistema busca administrar eficientemente productos, clientes, y pedidos, incorporando funcionalidades críticas como gestión de inventario, carrito de compras y facturación digital automática con cálculo de IVA (12%). La metodología propuesta es Scrum, con un plazo de 60 días para la entrega de un Producto Mínimo Viable (MVP). La arquitectura lógica se define en capas (N-Tier) para asegurar la escalabilidad y la integridad de los datos, especialmente la regla de no vender sin stock disponible. Las conclusiones resaltan la viabilidad y necesidad del proyecto para modernizar la cadena de valor de la zapatería, y se recomienda una preparación exhaustiva de los datos y una capacitación rigurosa del personal para garantizar la adopción exitosa del nuevo sistema.

Palabras Claves: E-commerce, Requerimientos, Scrum, Inventario, Facturación.

1. INTRODUCCIÓN:

El presente informe detalla la actividad de Análisis y Levantamiento de Requerimientos aplicada al caso de estudio de la Zapatería "Premium". Esta práctica es fundamental dentro del curso de Metodología de Desarrollo de Software, ya que permite la aplicación directa de técnicas de recolección de información, documentación y modelado de requisitos funcionales y no funcionales. El objetivo principal de esta actividad de laboratorio no solo es identificar las necesidades de digitalización de la zapatería, sino también formalizar la disciplina en el manejo de artefactos de desarrollo, como la definición clara de roles, reglas de negocio y alcance del proyecto. El contexto de la Zapatería "Premium" evidencia la necesidad de un sistema de gestión E-commerce para resolver problemas de ineficiencia operativa, lo cual establece el marco práctico para demostrar la comprensión de la fase inicial de un ciclo de vida de software.

2. OBJETIVO(S):

Objetivo General

El presente informe tiene como propósito principal analizar de manera integral el ecosistema del software libre y sus sistemas de licenciamiento, explorando los fundamentos filosóficos, técnicos y jurídicos que sustentan este modelo de desarrollo y distribución de software. Se busca comprender cómo las diferentes tipologías de licencias influyen en la accesibilidad, modificación y redistribución del código fuente, estableciendo un marco conceptual que permita identificar las implicaciones prácticas de adoptar licencias permisivas frente a licencias copyleft. Además, se pretende examinar el impacto socioeconómico que el software libre ha generado en la industria tecnológica contemporánea, analizando cómo este paradigma ha transformado las dinámicas de colaboración, innovación y competencia en el desarrollo de soluciones informáticas a nivel global.

Objetivos Específicos

Identificar y caracterizar los principios fundamentales de la filosofía del software libre, diferenciando los conceptos de libertad de uso, modificación, distribución y estudio del código fuente. Este objetivo busca establecer una base conceptual sólida que permita comprender las motivaciones éticas y técnicas que impulsan el movimiento del software libre, contrastándolo con modelos de software propietario y analizando cómo estas libertades fundamentales se materializan en la práctica del desarrollo colaborativo y la innovación tecnológica abierta.

Comparar y contrastar las características distintivas de las principales categorías de licencias de software libre, específicamente las licencias permisivas y las licencias copyleft, examinando sus requisitos legales, obligaciones de distribución y restricciones de uso. Se analizarán casos representativos como GPL, MIT y Apache para comprender cómo cada modelo de licenciamiento afecta la reutilización del código, la protección de la autoría y la sostenibilidad de proyectos de código abierto en contextos comerciales y no comerciales.

Evaluar el impacto socioeconómico del software libre en la industria tecnológica global, considerando aspectos como la reducción de costos de desarrollo, el fomento de la innovación colaborativa, la democratización del acceso a la tecnología y la creación de ecosistemas empresariales sustentables. Este análisis incluirá la revisión de modelos de negocio exitosos basados en software libre y su contribución al desarrollo tecnológico en diferentes regiones y sectores económicos.

3. MARCO TEÓRICO:

Fundamentos Filosóficos del Software Libre

El software libre representa un paradigma tecnológico y ético que trasciende la mera disponibilidad del código fuente, constituyéndose como un movimiento social que reivindica el derecho de los usuarios a controlar plenamente las herramientas informáticas que utilizan. La Free Software Foundation, fundada por Richard Stallman en 1985, estableció cuatro libertades esenciales que definen al software libre: la libertad de ejecutar el programa para cualquier propósito, la libertad de estudiar cómo

funciona el programa y modificarlo según las necesidades propias, la libertad de redistribuir copias para ayudar a otros, y la libertad de mejorar el programa y publicar dichas mejoras beneficiando a toda la comunidad (Salesforce, 2023). Estas libertades fundamentales se materializan a través del acceso irrestricto al código fuente, elemento técnico indispensable que permite la inspección, modificación y redistribución del software. La filosofía del software libre sostiene que el conocimiento tecnológico debe ser compartido libremente, promoviendo una ética de colaboración que contrasta radicalmente con el modelo de software propietario donde el código es mantenido como secreto comercial. Este enfoque colaborativo ha generado una cultura de desarrollo transparente donde miles de programadores de diferentes partes del mundo pueden contribuir simultáneamente a proyectos complejos, creando sistemas robustos y seguros mediante la revisión colectiva del código. La documentación técnica y los recursos educativos asociados al software libre también reflejan esta filosofía de apertura, permitiendo que cualquier persona con interés pueda aprender sobre tecnología sin barreras económicas o legales significativas.

Tipologías de Licencias de Software Libre

El ecosistema del software libre se caracteriza por una diversidad de esquemas de licenciamiento que, aunque comparten el compromiso fundamental con las libertades del usuario, difieren significativamente en sus requisitos legales y sus implicaciones para la distribución y comercialización del software. Las licencias permisivas, ejemplificadas por la MIT License y la Apache License, establecen requisitos mínimos para la redistribución del código, permitiendo que el software libre sea incorporado incluso en proyectos propietarios sin obligar a que las modificaciones sean liberadas bajo la misma licencia (Salesforce, 2023). Este enfoque flexible ha favorecido la adopción masiva de componentes de software libre en la industria comercial, facilitando la creación de productos híbridos que combinan código abierto con desarrollos propietarios. Por otro lado, las licencias copyleft, cuyo máximo exponente

es la GNU General Public License (GPL), establecen una condición viral que obliga a que cualquier obra derivada del software original mantenga las mismas libertades, garantizando que las mejoras y modificaciones permanezcan disponibles para la comunidad. Esta estrategia legal busca proteger el bien común del software libre contra su apropiación privada, asegurando que el conocimiento acumulado colectivamente no pueda ser restringido por entidades individuales. La elección entre licencias permisivas y copyleft refleja diferentes visiones sobre cómo equilibrar la libertad individual de los desarrolladores con el objetivo colectivo de mantener el software accesible para todos. Proyectos como Linux utilizan GPL para garantizar que el núcleo del sistema operativo permanezca libre, mientras que bibliotecas como React utilizan MIT para maximizar su adopción en diversos contextos comerciales.

Impacto Socioeconómico y Casos de Éxito

El software libre ha generado transformaciones profundas en la estructura económica de la industria tecnológica, desafiando modelos de negocio tradicionales basados en licencias de uso restringido y creando nuevas oportunidades para empresas que ofrecen servicios de soporte, personalización y consultoría alrededor de soluciones de código abierto. La reducción dramática en los costos de infraestructura tecnológica que posibilita el software libre ha democratizado el acceso a herramientas empresariales sofisticadas, permitiendo que organizaciones de menor tamaño y países en desarrollo puedan implementar sistemas de gestión de bases de datos, servidores web y aplicaciones empresariales sin las barreras financieras asociadas al software propietario (UVHM, 2020). Corporaciones tecnológicas líderes como IBM, Google y Microsoft han adoptado estrategias de contribución activa a proyectos de software libre, reconociendo que la colaboración abierta acelera la innovación y reduce los costos de investigación y desarrollo. El ecosistema de startups tecnológicas se ha beneficiado enormemente de la disponibilidad de componentes de código abierto que pueden ser integrados libremente en nuevos productos, reduciendo significativamente el tiempo y recursos necesarios para llevar innovaciones al mercado. Gobiernos de diversos países han implementado políticas de adopción preferente de software libre en la administración pública, motivados tanto por consideraciones económicas como por objetivos de soberanía tecnológica y seguridad informática. La comunidad académica utiliza extensivamente software libre para investigación científica y educación, facilitando la reproducibilidad de experimentos y eliminando barreras de acceso al conocimiento tecnológico. Este impacto multidimensional evidencia que el software libre no constituye únicamente una alternativa técnica al software propietario, sino un modelo

económico viable que promueve la innovación distribuida, la competencia saludable y el desarrollo tecnológico sostenible a escala global.

4. DESCRIPCIÓN DEL PROCEDIMIENTO:

El desarrollo de esta práctica de levantamiento de requerimientos y análisis de licencias se realizó mediante los siguientes materiales y métodos:

4.1. Materiales y Muestras

Muestra de Estudio: Zapatería "Premium" (Cliente).

Instrumentos: Grabadora digital o herramienta de videoconferencia para la entrevista; documentación base inicial (proporcionada por el cliente); herramientas ofimáticas para la transcripción y el análisis de la información.

Fuentes Teóricas: Bases de datos académicas y sitios oficiales (FSF, OSI) para el análisis comparativo de licencias.

4.2. Métodos Aplicados (Requerimientos de Zapatería)

Entrevista Estructurada: Se realizó una entrevista formal con los *stakeholders* (Diego Semanate y Andrea Sinmaleza) para determinar el objetivo principal del sistema, identificar los roles, la información crítica (ventas/pedidos) y los procesos esenciales.

Identificación de Requisitos: Se aplicó la técnica de diferenciación para separar las declaraciones del cliente en: Requisitos Funcionales (RF), Requisitos No Funcionales (RNF) y Reglas de Negocio (RN).

Modelado Conceptual: A partir de los datos críticos identificados, se definieron las entidades de datos (Producto, Pedido, Factura, Cliente) y sus atributos clave para iniciar el diseño de la base de datos.

Planificación del Alcance: Se definió el alcance del Producto Mínimo Viable (MVP) y un cronograma tentativo de Sprints (60 días, 4 Sprints), aplicando principios de la metodología Scrum.

SECCIÓN DE REQUERIMIENTOS:

Artefacto	Título Recomendado en el Informe	Ubicación	Razón
Plan de Riesgos Iniciales	Gestión de Riesgos Iniciales	Dentro de la sección de Metodología o Requerimientos, antes del análisis detallado.	Muestra que evaluaste proactivamente las amenazas al proyecto (como el *Scope Creep* o la migración de datos).
Plan de Gestión de Cambios	Proceso de Gestión de Cambios (PGC)	Inmediatamente después del Plan de Riesgos.	Documenta la metodología ágil (*Scrum*) y cómo el proyecto mantendrá el control sobre las desviaciones de alcance.

5. ANÁLISIS DE RESULTADOS:

5.1. Análisis Funcional y Roles

El sistema contará con tres roles diferenciados y sus respectivas funcionalidades:

Rol	Descripción	Permisos Clave
Administrador	Gestor principal.	Gestión total de productos, stock, precios, y usuarios.
Usuario (Empleado)	Personal de tienda.	Gestión de pedidos y verificación de inventario.
Cliente	Comprador final.	Compras en línea, uso del carrito, visualización de pedidos.

Requerimientos Funcionales (RF)

Código	Descripción	Módulo Principal
RF1	Registrar y actualizar productos.	Gestión de Productos
RF2	Permitir compras en línea con carrito virtual.	E-commerce / Ventas
RF3	Generar facturas digitales automáticamente.	Facturación
RF4	Actualizar el stock tras cada venta o devolución.	Gestión de Inventario
RF5	Gestionar usuarios y roles.	Gestión de Usuarios
RF6	Mostrar el estado y la trazabilidad de los pedidos.	Seguimiento de Pedidos

5.2. Reglas de Negocio (RN) y Datos

RN1 (Código Único): Cada producto debe tener un código único (SKU).

RN2 (Stock Crítico): No se permite procesar ventas si el stock disponible es cero.

RN3 (Cálculo IVA): El IVA (12%) se calculará automáticamente en el total de la venta.

Entidades de Datos

Entidad	Atributos Clave	Restricciones
Producto	código (PK), nombre, talla, precio, stock	código (Único), stock (>= 0).
Cliente	idCliente (PK), nombre, dirección, teléfono, correo	correo (Acceso, Único).
Pedido	idPedido (PK), fecha, cliente (FK), total, estado	total (Calculado con IVA).
Factura	número (PK), fecha, cliente (FK), total, pedidoId (FK)	número (Único), Generación obligatoria (RF3).

Roles y Permisos

Rol	Descripción	Permisos Clave
Administrador	Gestor principal.	Gestión total de productos, stock, precios, y usuarios.
Usuario (Empleado)	Personal de tienda.	Gestión de pedidos y verificación de inventario.
Cliente	Comprador final.	Compras en línea, uso del carrito, visualización de pedidos.

Glosario del Dominio

- Cliente: Persona que compra productos. Tiene datos de contacto y estado activo/inactivo.

- Usuario: Cuenta de acceso al sistema. Puede tener rol ADMIN o VENDEDOR.

- Administrador: Usuario con permisos para gestionar usuarios y configuración global.

- Vendedor: Usuario que registra pedidos y gestiona clientes/productos en la operación diaria.

- Producto: Ítem del catálogo (nombre, descripción, categoría, precio, stock, activo).

- Inventario/Stock: Cantidad disponible de un producto en almacén.

- Pedido: Orden de compra registrada a un cliente; contiene fecha, estado y total.

- Estado de Pedido: Ciclo de vida: REGISTRADO, PAGADO, ENTREGADO, ANULADO.

- Detalle de Pedido: Línea del pedido con producto, cantidad, precio unitario, subtotal.

- Subtotal: cantidad × precioUnitario de una línea.

- Total: Suma de subtotales del pedido.

- Catálogo: Conjunto de productos disponibles para venta.

- Registro de venta: Proceso de creación de un pedido con sus detalles.

- Autenticación: Verificación de credenciales del usuario para iniciar sesión.

- Autorización: Validación de permisos según rol (ADMIN/VENDEDOR).

- Cliente activo: Cliente habilitado para realizar pedidos.

- Producto activo: Producto visible para venta.

- Reserva/Descuento de stock: Reducción de stock al confirmar el pedido.

- Anulación: Cambio de estado del pedido a ANULADO (puede revertir stock).

- Reporte de ventas: Resumen de pedidos/ingresos en un período.

- Repositorio: Capa de acceso a datos (persistencia).

- Controlador: Capa que expone endpoints/acciones de aplicación.

- Validación de stock: Regla que exige stock >= cantidad por producto al registrar un pedido.

Casos de Uso (Resumen)

ID	Caso de uso	Actor(es)	Objetivo	Resultado
UC01	Autenticarse	Vendedor, Administrador	Ingresar al sistema con credenciales válidas	Sesión iniciada o error
UC02	Gestionar Productos (CRUD)	Administrador	Crear/editar/listar/desactivar productos	Catálogo actualizado
UC03	Gestionar Clientes (CRUD)	Vendedor, Administrador	Crear/editar/listar clientes	Clientes actualizados
UC04	Registrar Pedido	Vendedor	Registrar una venta a un cliente con uno o más productos	Pedido REGISTRADO, stock descontado
UC05	Pagar Pedido	Vendedor	Registrar el pago de un pedido	Estado PAGADO
UC06	Consultar Pedidos	Vendedor, Administrador, Cliente	Buscar/filtrar pedidos por fecha/cliente/estado	Lista de pedidos
UC07	Actualizar Stock	Vendedor, Administrador	Ajustar stock (corrección o ingreso)	Stock actualizado
UC08	Gestionar Usuarios (CRUD)	Administrador	Crear/editar/desactivar usuarios y roles	Usuarios actualizados
UC09	Generar Reporte de Ventas	Administrador	Obtener totales por período	Reporte generado

Caso de Uso Extendido: UC04 - Registrar Pedido

Nombre: Registrar Pedido

Actores: Vendedor (primario); Repositorio de Productos/Clientes/Pedidos (secundarios)

Propósito: Registrar una venta asociada a un cliente y descontar stock.

Alcance: Sistema Zapatería (backend + UI)

Nivel: Usuario final

Disparador: Vendedor inicia una nueva venta en la UI.

Precondiciones

- El Vendedor está autenticado (UC01 completado).

- El Cliente existe y está activo (o se dará de alta durante el flujo).

- Los productos a vender están activos.

Postcondiciones

- Éxito: Pedido creado con estado REGISTRADO, fecha y total calculado; stock descontado.

- Falla: No se crea pedido ni se modifica stock.

Reglas de Negocio

- RB1: subtotal = cantidad × precioUnitario.

- RB2: total = suma de subtotales.

- RB3: stockDisponible >= cantidad.

- RB4: Estado inicial REGISTRADO.

- RB5: Fecha del pedido = fecha del sistema.

Flujo Principal

1. El Vendedor selecciona 'Nuevo Pedido'.

2. El sistema solicita cliente y productos.

3. El Vendedor selecciona cliente.

4. El Vendedor agrega líneas con productos y cantidades.

5. El sistema valida stock.

6. Calcula subtotales y total.

7. El Vendedor confirma pedido.

8. El sistema guarda pedido y descuenta stock.

9. El sistema muestra número y total del pedido.

Flujos Alternos

- A1 – Cliente no existe: crear cliente y continuar.

- A2 – Modificar cantidades: recalcula totales.

- A3 – Producto inactivo: reemplazar o eliminar producto.

Excepciones

- E1 – Stock insuficiente: Notifica y permite ajustar.

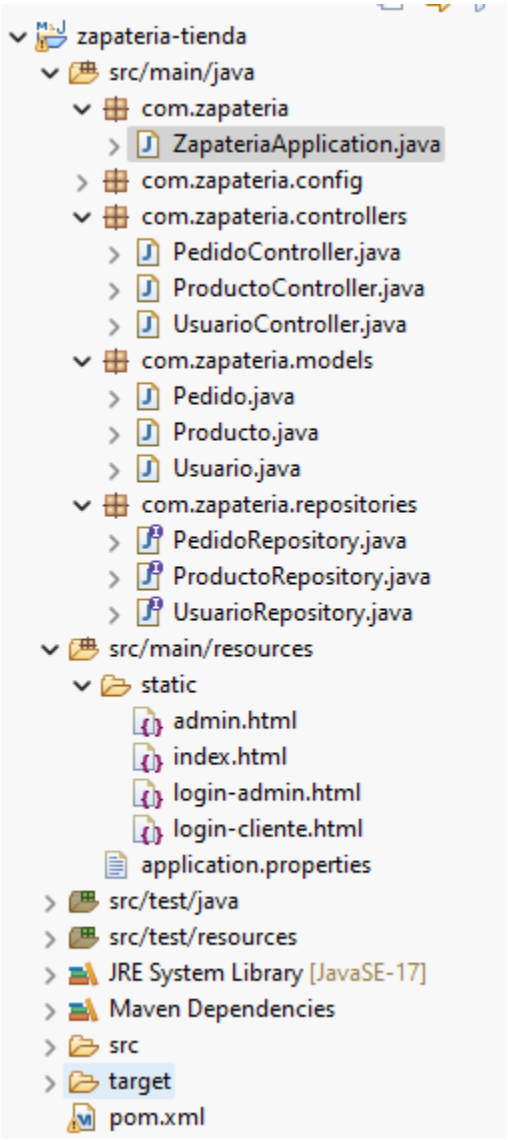
- E2 – Error de persistencia: Revierte transacción.

- E3 – Cliente inactivo: Notifica y ofrece activarlo.

6. GRÁFICOS O FOTOGRAFÍAS:

Licencia	Tipo	Copyleft	Requisitos de Atribución	Distribución de Código Fuente	Compatibilidad Comercial	Obligaciones de Modificaciones
GNU GPL v3	Copyleft fuerte	Sí	Obligatorio mencionar autoría original y cambios realizados	Obligatorio distribuir código fuente con cualquier distribución binaria	Permitido, pero modificaciones deben ser GPL	Modificaciones deben liberarse bajo GPL si se distribuyen
MIT License	Permisiva	No	Obligatorio incluir aviso de copyright y licencia	No obligatorio, opcional	Totalmente compatible, permite uso en software propietario	Sin obligación de liberar modificaciones
Apache License 2.0	Permisiva	No	Obligatorio incluir aviso de licencia y atribución	No obligatorio, opcional	Totalmente compatible con uso comercial	Sin obligación, pero cambios deben documentarse

Estructura:


zapateria-tienda
src/main/java
com.zapateria
ZapateriaApplication.java
com.zapateria.config
com.zapateria.controllers
PedidoController.java
ProductoController.java
UsuarioController.java
com.zapateria.models
Pedido.java
Producto.java
Usuario.java
com.zapateria.repositories
PedidoRepository.java
ProductoRepository.java
UsuarioRepository.java
src/main/resources
static
admin.html
index.html
login-admin.html
login-cliente.html
application.properties
src/test/java
src/test/resources
JRE System Library [JavaSE-17]
Maven Dependencies
src
target
pom.xml

Código:

Ilustración 1 Main

```
package com.zapateria;

import org.springframework.boot.SpringApplication;

@SpringBootApplication
public class ZapateriaApplication {

    public static void main(String[] args) {
        SpringApplication.run(ZapateriaApplication.class, args);

        System.out.println("\n╔═══════════════════════════════════════╗");
        System.out.println("║   Sistema de Zapatería - Versión Simple   ║");
        System.out.println("║   Servidor iniciado correctamente         ║");
        System.out.println("║   URL: http://localhost:8080              ║");
        System.out.println("╚═══════════════════════════════════════╝\n");

        System.out.println("📋 Endpoints disponibles:");
        System.out.println("  • POST /api/usuarios/login        - Login");
        System.out.println("  • POST /api/usuarios/registro     - Registro");
        System.out.println("  • GET  /api/productos             - Listar productos");
        System.out.println("  • POST /api/productos             - Crear producto");
        System.out.println("  • GET  /api/clientes              - Listar clientes");
        System.out.println("  • POST /api/pedidos               - Crear pedido");
        System.out.println();
    }
}
```

Ilustración 2 WebConfig

```
package com.zapateria.config;

import org.springframework.context.annotation.Configuration;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        // Configurar recursos estáticos
        registry.addResourceHandler("/**")
                .addResourceLocations("classpath:/static/")
                .setCachePeriod(0);

        System.out.println("☑ Recursos estáticos configurados: classpath:/static/");
    }

    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        // Redirigir la raíz al index.html
        registry.addViewController("/").setViewName("forward:/index.html");

        System.out.println("☑ Vista raíz configurada: / -> /index.html");
    }

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        // Configurar CORS
```

Ilustración 3 PedidoController

```java
package com.zapateria.controllers;
import java.util.HashMap;

@RestController
@RequestMapping("/api/pedidos")
@CrossOrigin(origins = "*")
public class PedidoController {

    @Autowired
    private PedidoRepository pedidoRepository;

    @Autowired
    private ProductoRepository productoRepository;

    // Listar todos los pedidos
    @GetMapping
    public List<Pedido> listarPedidos() {
        return pedidoRepository.findAll();
    }

    // Obtener pedido por ID
    @GetMapping("/{id}")
    public Pedido obtenerPedido(@PathVariable String id) {
        return pedidoRepository.findById(id).orElse(null);
    }

    // Pedidos por cliente
    @GetMapping("/cliente/{clienteId}")
    public List<Pedido> pedidosPorCliente(@PathVariable String clienteId) {
```

Ilustración 4 ProductoController

```java
package com.zapateria.controllers;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.zapateria.models.Producto;
import com.zapateria.repositories.ProductoRepository;

@RestController
@RequestMapping("/api/productos")
@CrossOrigin(origins = "*")
public class ProductoController {

    @Autowired
    private ProductoRepository productoRepository;

    // Listar todos los productos
```

Ilustración 5 UsuarioController

```java
package com.zapateria.controllers;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.zapateria.models.Usuario;
import com.zapateria.repositories.UsuarioRepository;

@RestController
@RequestMapping("/api/usuarios")
@CrossOrigin(origins = "*")
public class UsuarioController {

    @Autowired
    private UsuarioRepository usuarioRepository;

    // Login
    @PostMapping("/login")
    public Map<String, Object> login(@RequestBody Usuario loginData) {
        Map<String, Object> response = new HashMap<>();

        Usuario usuario = usuarioRepository.findByUsernameAndPassword(
            loginData.getUsername(),
            loginData.getPassword()
        );
```

Ilustración 6 Pedidos models

```java
package com.zapateria.models;
import java.util.Date;
import java.util.List;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "pedidos")
public class Pedido {
    @Id
    private String id;
    private String numeroPedido;
    private String clienteId;
    private String clienteNombre;
    private List<ItemPedido> items;
    private double total;
    private String estado; // "PENDIENTE", "PROCESANDO", "ENVIADO", "ENTREGADO"
    private Date fecha;

    // Constructor vacío
    public Pedido() {
        this.fecha = new Date();
        this.estado = "PENDIENTE";
    }

    // Getters y Setters
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
```

Ilustración 7 Producto - Models

```java
package com.zapateria.models;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "productos")
public class Producto {
    @Id
    private String id;
    private String codigo;
    private String nombre;
    private String descripcion;
    private String marca;
    private String categoria;
    private String talla;
    private String color;
    private double precio;
    private int stock;

    // Constructor vacío
    public Producto() {}

    // Getters y Setters
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getCodigo() { return codigo; }
    public void setCodigo(String codigo) { this.codigo = codigo; }
```

Ilustración 8 Usuario - Models

```java
package com.zapateria.models;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "usuarios")
public class Usuario {
    @Id
    private String id;
    private String nombre;
    private String apellido;
    private String email;
    private String username;
    private String password;
    private String telefono;
    private String direccion;
    private String rol; // "CLIENTE" o "ADMINISTRADOR"

    public Usuario() {}

    // Getters y Setters
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getApellido() { return apellido; }
    public void setApellido(String apellido) { this.apellido = apellido; }
```

Ilustración 9 PedidoRepository - Repositories

```java
package com.zapateria.repositories;

import java.util.List;

@Repository
public interface PedidoRepository extends MongoRepository<Pedido, String> {
    // Buscar pedidos por cliente
    List<Pedido> findByClienteId(String clienteId);

    // Buscar pedidos por estado
    List<Pedido> findByEstado(String estado);

    // Buscar pedido por número
    Pedido findByNumeroPedido(String numeroPedido);
}
```

Ilustración 10 ProductoRepository - Repositories

```java
package com.zapateria.repositories;

import java.util.List;

@Repository
public interface ProductoRepository extends MongoRepository<Producto, String> {
    // Buscar productos por categoría
    List<Producto> findByCategoria(String categoria);

    // Buscar productos por marca
    List<Producto> findByMarca(String marca);

    // Buscar productos por nombre (contiene texto)
    List<Producto> findByNombreContainingIgnoreCase(String nombre);

    // Buscar productos con stock bajo
    List<Producto> findByStockLessThan(int stock);
}
```

Ilustración 11 UsuarioRepository - Repositories

```java
package com.zapateria.repositories;

import org.springframework.data.mongodb.repository.MongoRepository;

@Repository
public interface UsuarioRepository extends MongoRepository<Usuario, String> {
    Usuario findByUsername(String username);
    Usuario findByUsernameAndPassword(String username, String password);
    Usuario findByEmail(String email);
}
```

Ilustración 12 Admin

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Panel Administrativo - Zapatería</title>
    <script src="https://unpkg.com/lucide@latest"></script>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        :root {
            --primary: #f59e0b;
            --primary-dark: #d97706;
            --success: #10b981;
            --danger: #ef4444;
            --warning: #f59e0b;
            --info: #3b82f6;
            --dark: #1f2937;
            --light: #f3f4f6;
        }

        body {
            font-family: 'Inter', 'Segoe UI', sans-serif;
            background: #f9fafb;
            display: flex;
```

Ilustración 13 Index

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Panel Administrativo - Zapatería</title>
    <script src="https://unpkg.com/lucide@latest"></script>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        :root {
            --primary: #f59e0b;
            --primary-dark: #d97706;
            --success: #10b981;
            --danger: #ef4444;
            --warning: #f59e0b;
            --info: #3b82f6;
            --dark: #1f2937;
            --light: #f3f4f6;
        }

        body {
            font-family: 'Inter', 'Segoe UI', sans-serif;
            background: #f9fafb;
            display: flex;
```

Ilustración 14 Login-admin

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Administrador - Zapatería</title>
    <script src="https://unpkg.com/lucide@latest"></script>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        :root {
            --primary: #f59e0b;
            --primary-dark: #d97706;
            --success: #10b981;
            --danger: #ef4444;
            --dark: #1f2937;
        }

        body {
            font-family: 'Inter', 'Segoe UI', sans-serif;
            background: linear-gradient(135deg, #f59e0b 0%, #dc2626 100%);
            min-height: 100vh;
            display: flex;
            align-items: center;
```

Ilustración 15 Login - Usuario

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Cliente - Zapatería</title>
    <script src="https://unpkg.com/lucide@latest"></script>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        :root {
            --primary: #6366f1;
            --primary-dark: #4f46e5;
            --success: #10b981;
            --danger: #ef4444;
            --dark: #1f2937;
            --light: #f3f4f6;
        }

        body {
            font-family: 'Inter', 'Segoe UI', sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            display: flex;
```

Ilustración 16 Inicio de sesion - Usuario

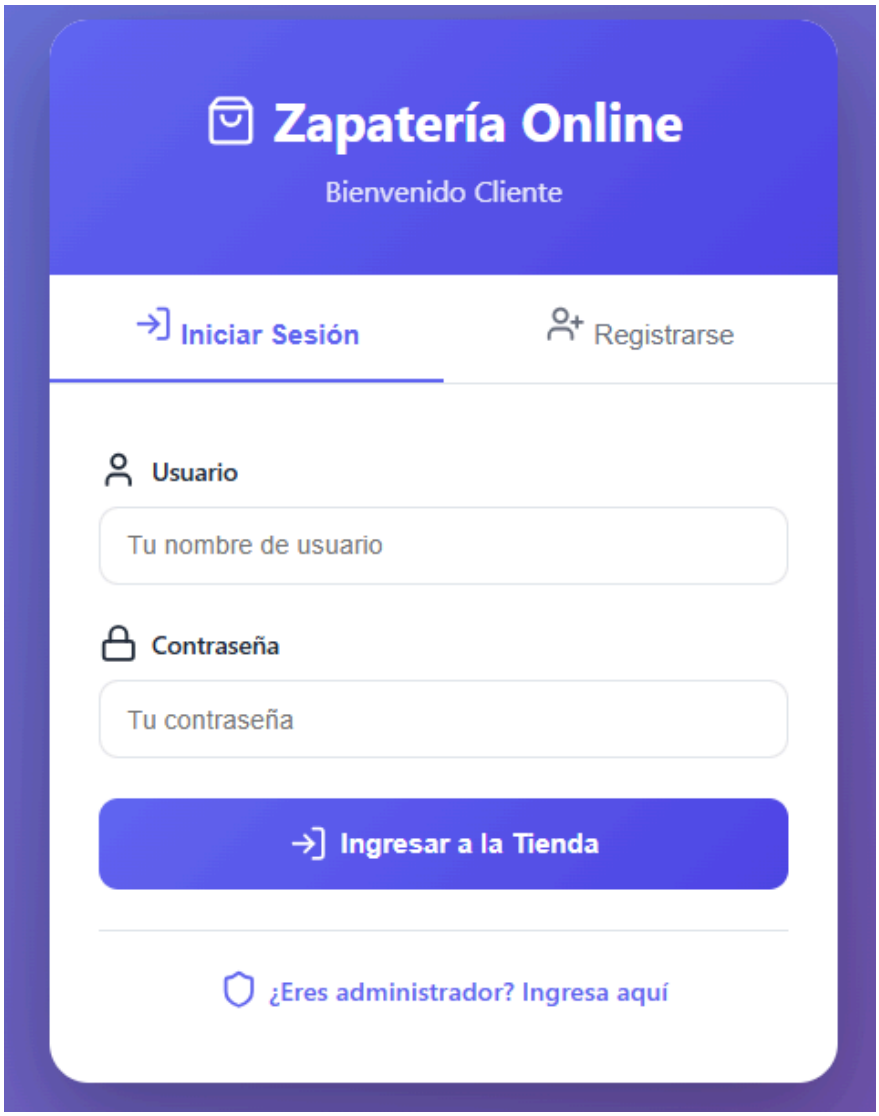


Ilustración 17 Inicio de sesion - administrador

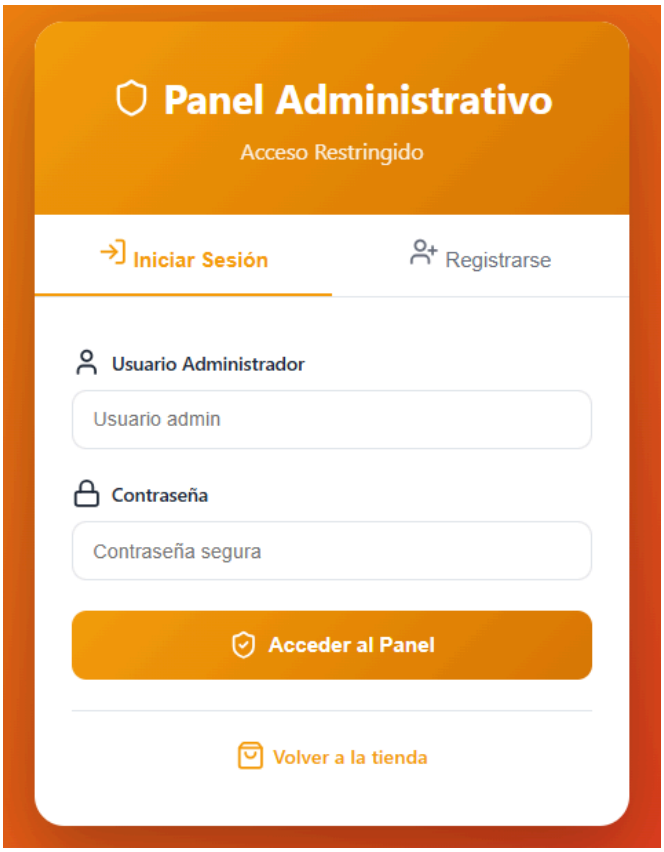


Ilustración 18 Administrador-Home

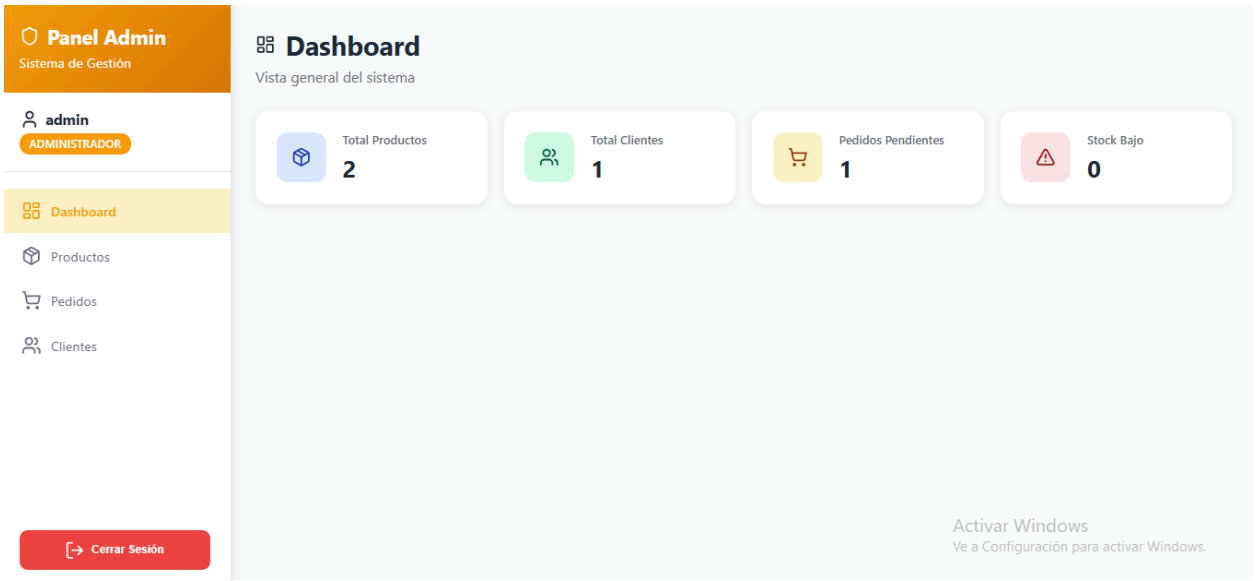


Ilustración 19 Agregar Productos

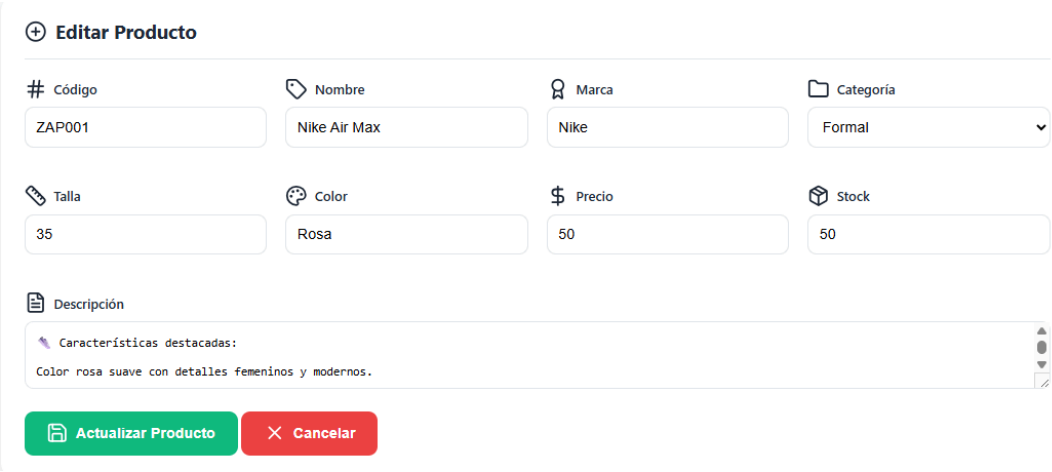


Ilustración 20 Lista de Productos - Administrador

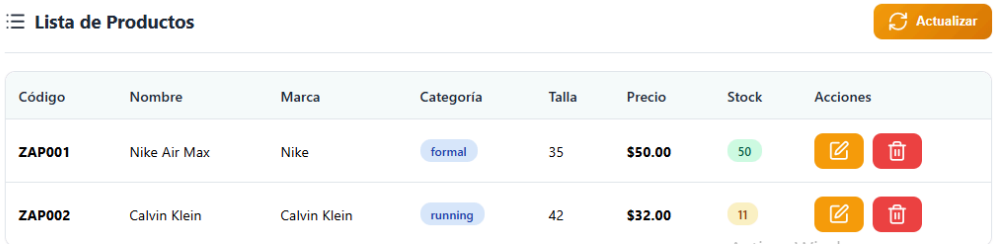


Ilustración 21 Gestion de Pedidos - Administrador

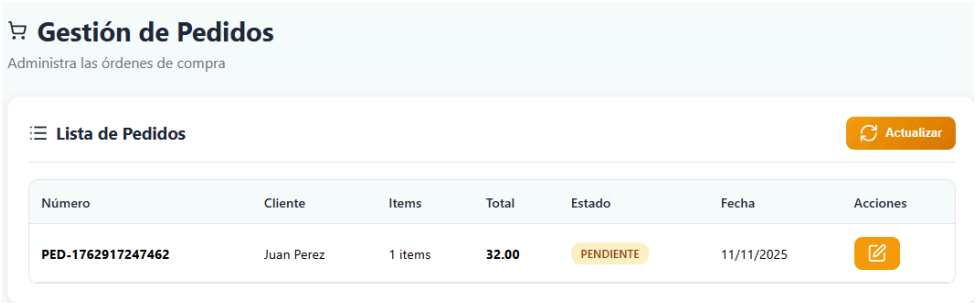


Ilustración 22 Lista de Clientes - Administrador

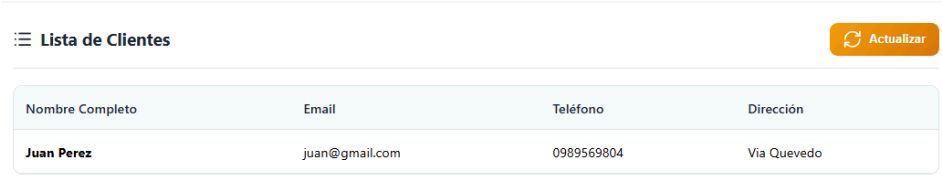


Ilustración 23MongoDB - Usuarios


```
_id: ObjectId('6913f0f6924eeb0c8570142c')
nombre : "Juan"
apellido : "Perez"
email : "juan@gmail.com"
username : "juan02"
password : "Juan02*"
telefono : "0989569804"
direccion : "Via Quevedo"
rol : "CLIENTE"
_class : "com.zapateria.models.Usuario"
```

```
_id: ObjectId('6913f99c924eeb0c8570142d')
nombre : "admin"
apellido : "admin"
email : "admin@example.com"
username : "admin"
password : "123456"
rol : "ADMINISTRADOR"
_class : "com.zapateria.models.Usuario"
```


Ilustración 24 MongoDB-Productos


```
_id: ObjectId('6913fa33924eeb0c8570142e')
codigo : "ZAP001"
nombre : "Nike Air Max"
descripcion : "🪶 Características destacadas:

              Color rosa suave con detalles femenino…"
marca : "Nike"
categoria : "formal"
talla : "35"
color : "Rosa"
precio : 50
stock : 50
_class : "com.zapateria.models.Producto"
```

```
_id: ObjectId('6913fac2924eeb0c8570142f')
codigo : "ZAP002"
nombre : "Calvin Klein"
descripcion : "📐 Características principales:

              Color morado intenso y elegante, idea…"
marca : "Calvin Klein"
categoria : "running"
talla : "42"
```


50

Activar Windows
Ve a Configuración para activar Windows

Ilustración 25 Panel de Inicio - Usuario

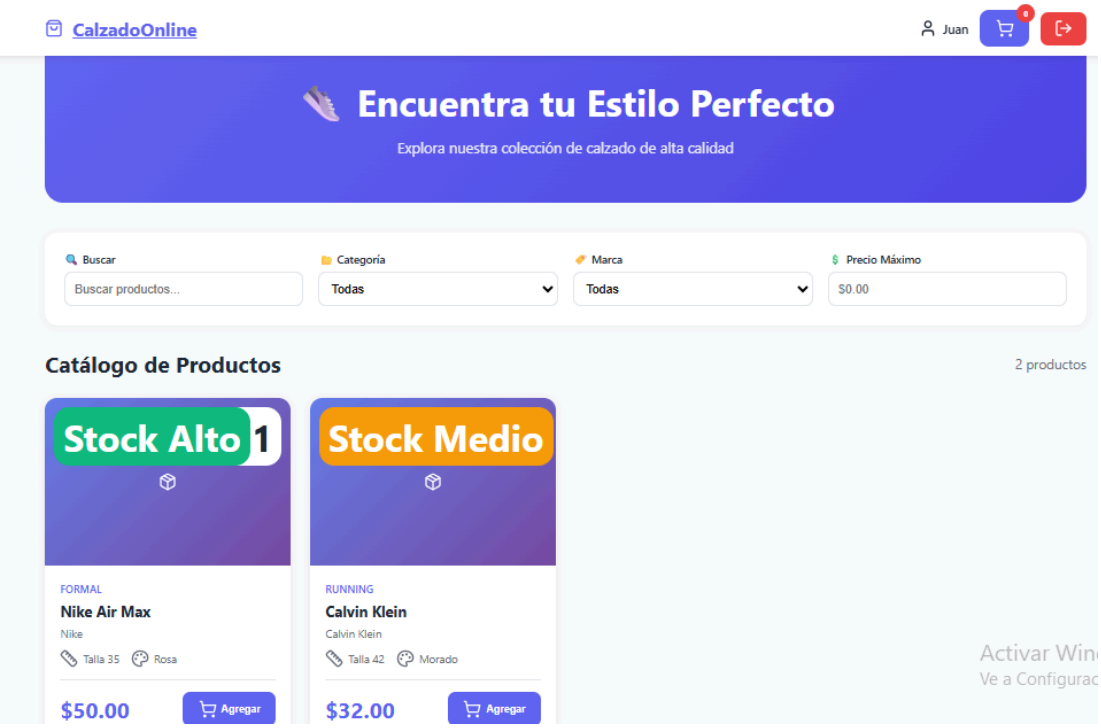


Ilustración 26 Mi Carrito - Usuario



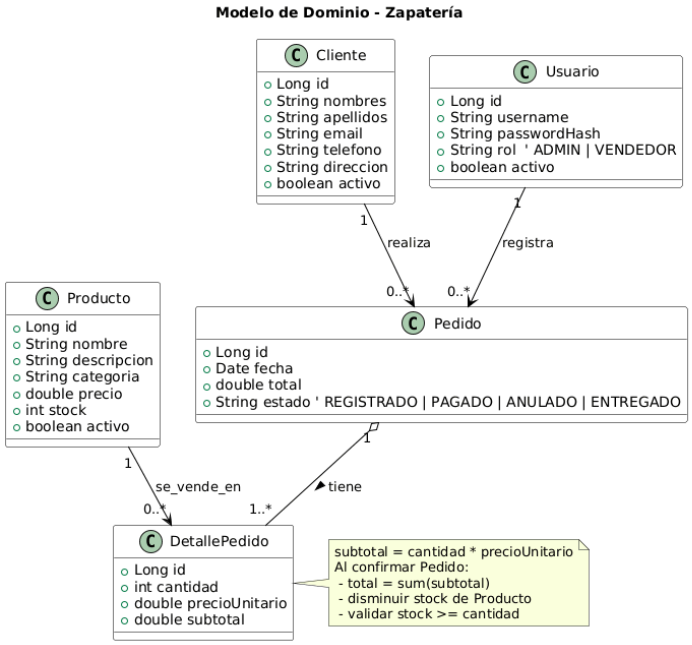


Ilustración 27 Diagrama UML

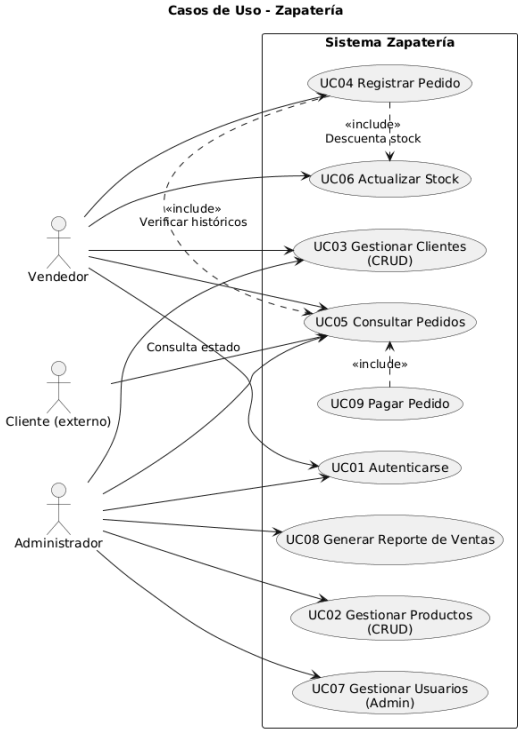


Ilustración 28 Diagrama de Casos de Uso (refinado)

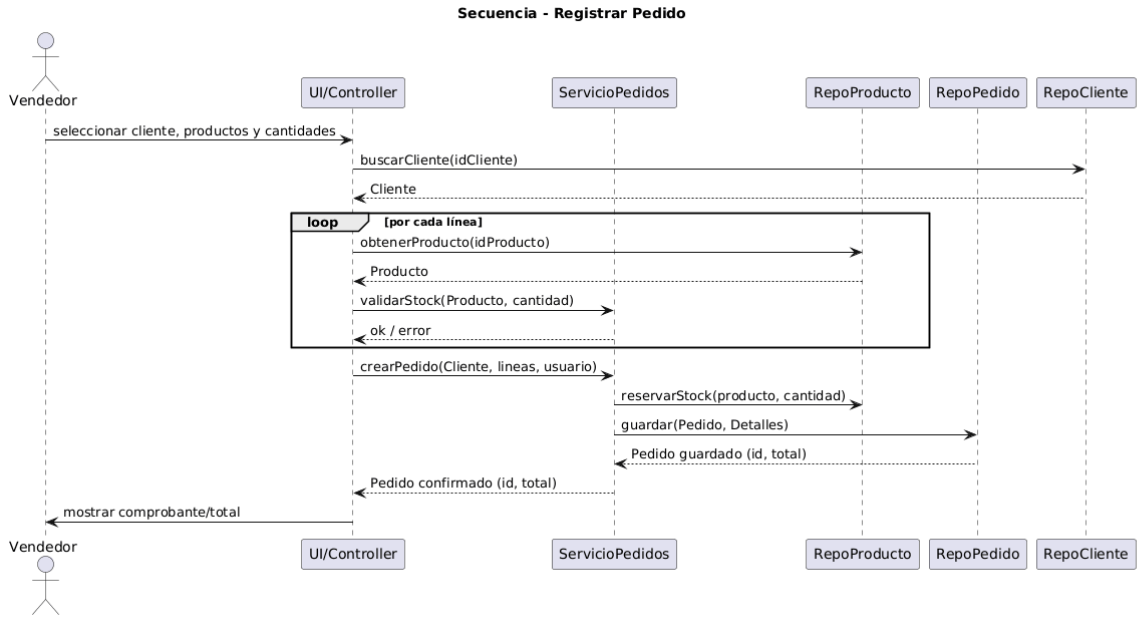


Ilustración 29 Escenario 1: Registrar pedido (venta) — Secuencia

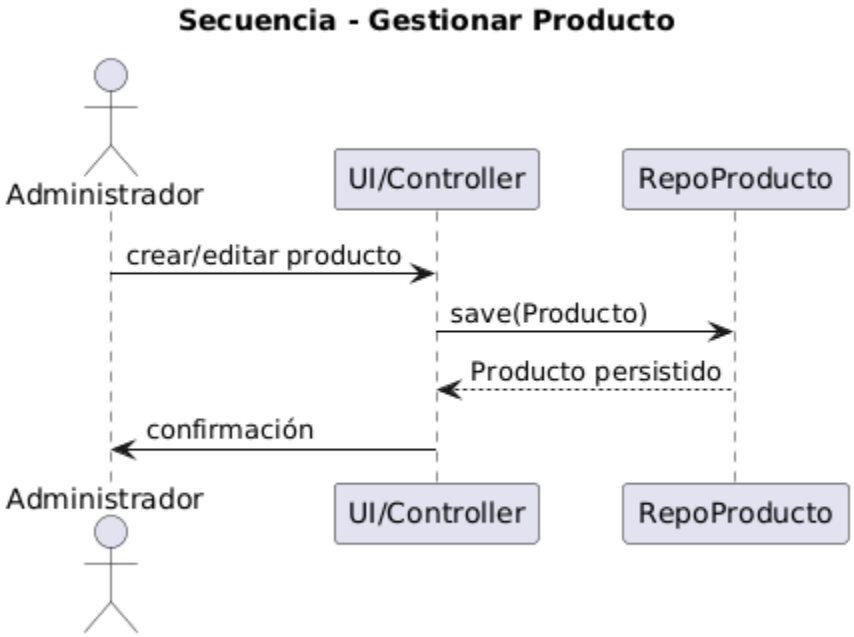


Ilustración 30 Escenario 2: Alta/edición de producto — Secuencia

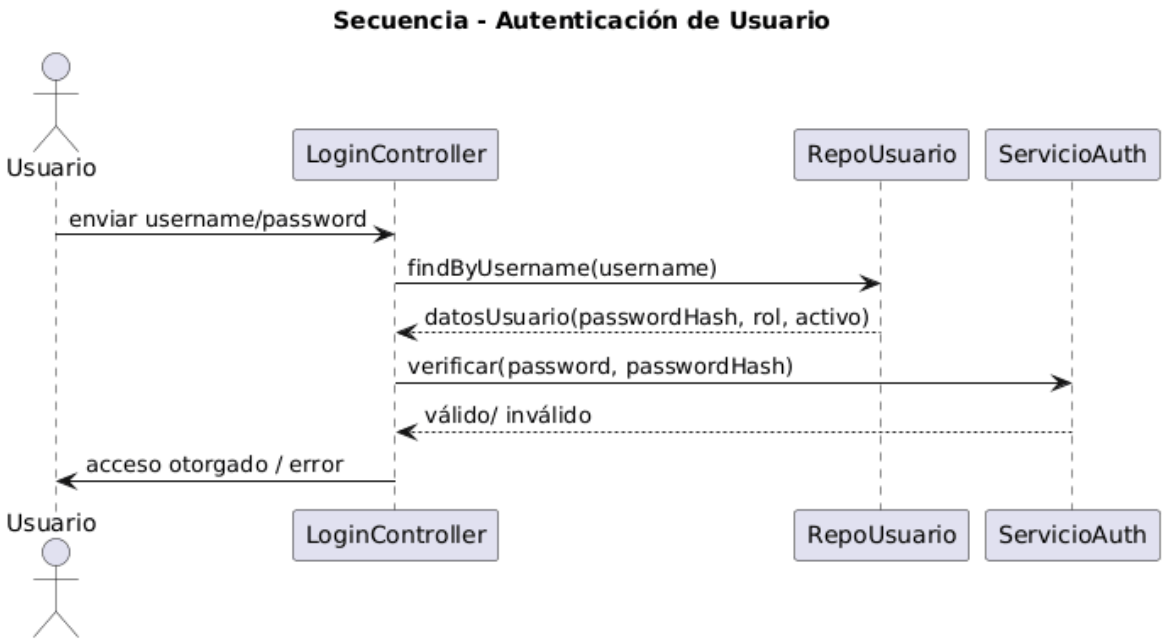


Ilustración 30 Escenario 3: Autenticación

7. DISCUSIÓN:

Al iniciar la discusión, compruebo que el análisis de requerimientos para la Zapatería "Premium" es el paso fundamental para la transición hacia una plataforma de E-commerce. Los artefactos que desarrollé, como los Requerimientos Funcionales (RF) y las Reglas de Negocio (RN) , me permitieron aplicar directamente las técnicas de recolección y modelado de requisitos que aprendí en el curso de Metodología de Desarrollo de Software. Por ejemplo, la regla de negocio RN2 —no permitir ventas sin stock — junto con la funcionalidad de actualizar el stock tras cada venta (RF4) , son esenciales para asegurar la integridad del inventario y la viabilidad operativa del sistema, que es el propósito principal del proyecto.

En cuanto al marco teórico, mi análisis sobre el software libre me sirvió para establecer una base conceptual sólida. Entendí que la filosofía se centra en las cuatro libertades esenciales definidas por la Free Software Foundation , que son la libertad de uso, estudio, redistribución y mejora del código fuente. Al profundizar en las tipologías de licencias, confirmé la diferencia crítica entre las licencias Copyleft como GPL , que protegen el bien común al obligar a que las modificaciones se liberen bajo la misma licencia , y las licencias Permisivas como MIT o Apache , que fomentan una mayor adopción en proyectos comerciales, incluso propietarios. Este contraste me hace reflexionar sobre la flexibilidad que un proyecto de código abierto ofrece al equilibrar la libertad de los desarrolladores con el objetivo colectivo de mantener el software accesible. Finalmente, la documentación me confirmó que el impacto socioeconómico del software libre va más allá de lo técnico; es un modelo que democratiza el acceso a la tecnología y fomenta la innovación colaborativa , lo cual es crucial para modernizar la cadena de valor de negocios como la zapatería.

8. CONCLUSIONES:

La filosofía del software libre se fundamenta en principios éticos que trascienden el ámbito puramente tecnológico, estableciendo un marco conceptual donde la libertad de acceso, modificación y distribución del código fuente constituyen derechos fundamentales de los usuarios. Estos principios han demostrado ser catalizadores efectivos de innovación tecnológica colaborativa, generando ecosistemas de desarrollo donde la transparencia y la revisión colectiva producen software de alta calidad y seguridad. La materialización práctica de estas libertades a través del acceso irrestricto al código fuente ha permitido que comunidades globales de desarrolladores contribuyan simultáneamente a proyectos complejos, superando las limitaciones de modelos de desarrollo centralizados. El contraste con el software propietario evidencia que el control del conocimiento tecnológico tiene implicaciones directas sobre la autonomía de los usuarios y la distribución equitativa de oportunidades de aprendizaje y desarrollo profesional en el ámbito informático.

Las diferentes categorías de licencias de software libre, particularmente la distinción entre licencias permisivas y copyleft, reflejan concepciones divergentes sobre cómo proteger y promover las libertades del software libre en contextos comerciales y de desarrollo colaborativo. Las licencias permisivas como MIT y Apache han facilitado la adopción masiva de componentes de código abierto en productos comerciales, maximizando el impacto y la difusión del software libre a costa de permitir su incorporación en sistemas propietarios. Por otro lado, licencias copyleft como GPL garantizan que las mejoras y modificaciones permanezcan accesibles para la comunidad, protegiendo el bien común tecnológico contra su apropiación privada mediante mecanismos legales que obligan a mantener las libertades originales en obras derivadas. Esta diversidad de esquemas de licenciamiento ofrece a desarrolladores y organizaciones la flexibilidad de elegir modelos que se alineen con sus objetivos específicos, ya sean estos la máxima difusión tecnológica o la preservación rigurosa de las libertades fundamentales del software libre.

El impacto socioeconómico del software libre ha transformado fundamentalmente la estructura de la industria tecnológica global, democratizando el acceso a herramientas sofisticadas y generando

modelos de negocio innovadores basados en servicios de valor agregado en lugar de licencias restrictivas de uso. La reducción de barreras económicas para acceder a tecnología empresarial ha permitido que organizaciones de menor tamaño y regiones en desarrollo implementen infraestructuras tecnológicas robustas, contribuyendo a la reducción de la brecha digital global. Corporaciones tecnológicas líderes han reconocido que la colaboración abierta acelera significativamente la innovación y reduce costos de investigación, integrando estratégicamente proyectos de código abierto en sus ecosistemas comerciales. La adopción gubernamental de software libre en administraciones públicas responde tanto a consideraciones de eficiencia económica como a objetivos estratégicos de soberanía tecnológica y seguridad informática, evidenciando que este paradigma constituye una alternativa viable y sostenible frente a modelos de software propietario tradicionales.

9. BIBLIOGRAFÍA:

Codigonautas. (s.f.). Qué es HTML. https://codigonautas.com/que-es-html/

Salesforce. (2023). Definition API. https://www.salesforce.com/es/resources/articles/definition-api/

Universidad Virtual del Estado de Hidalgo. (2020). Bases de Datos. https://bdigital.uvhm.edu.mx/wp-content/uploads/2020/05/Bases-de-Datos.pdf

Flipbook. (n.d.). Flipbookpdf.net. Retrieved November 10, 2025, from https://www.flipbookpdf.net/web/site/36cb19597af525883ebca02b06834c5271ed80dd202511.pdf.html#page/1